

Universidad Técnica Estatal de Quevedo (UTEQ)

Facultad de Ciencias de la Computación

Carrera de Ingeniería de Software (Rediseño)

Asignatura: Aplicaciones Web — Período académico 2026–2027

Sistema de Gestión Bibliotecaria Web (SGB-SaaS)

Entrega 1B — Módulo de Autenticación JWT y Acceso a Datos
con Spring Data JPA

Java 21 / Spring Boot 4

Angular 17

PostgreSQL 16

Redis 7

OWASP

Integrantes:

Cajas Ibarra Irvin Marcelo	C.I.: 1251039606
Loor Medranda Marlon Taylor	C.I.: 0928087469
Panama Murillo Moises Antonio	C.I.: 1251334544

Docente:

Ing. Gleiston Cicerón Guerrero Ulloa

Fecha de entrega:

20 de Junio de 2026

Repositorio:

<https://github.com/mloorm14/sgb-saas> — Rama: entrega-1b

Resumen ejecutivo

En esta entrega se implementaron los dos componentes centrales del Sistema de Gestión Bibliotecaria Web (SGB-SaaS): el módulo de autenticación stateless basado en JSON Web Tokens (JWT) y el módulo de acceso a datos para la entidad **Libro** mediante Spring Data JPA. El sistema resuelve la falta de control centralizado del inventario y de los procesos de identificación de usuarios en bibliotecas institucionales ecuatorianas, problemática identificada en la Entrega 1A. Se utilizó el stack Java 21 LTS con Spring Boot 4.0.6, Spring Security 7 y la biblioteca jjwt 0.12 para la emisión y validación de tokens firmados con HS256. La persistencia se gestionó con Hibernate 6 sobre PostgreSQL 16, con el esquema versionado mediante Flyway 9, lo que garantiza reproducibilidad entre entornos. Para mitigar el riesgo de reutilización de tokens tras el cierre de sesión, se incorporó Redis 7 como almacén de una lista negra de identificadores JWT (JTI) con expiración (TTL) sincronizada con la del token. El frontend Angular 17 implementa un interceptor HTTP que adjunta el token desde memoria (sin uso de `localStorage`) y un `AuthGuard` que protege las rutas privadas. Se desarrollaron once pruebas automatizadas con JUnit 5 y Mockito (cinco de autenticación, cinco de CRUD de Libro y una de integración de contexto), todas en estado satisfactorio, y se documentaron los endpoints con springdoc-openapi en `/swagger-ui.html`. Las métricas registradas muestran un tiempo promedio de respuesta del endpoint de login de 210 ms (P95: 260 ms), con una reducción aproximada del 70 % en el tiempo de respuesta del listado de libros al habilitar la caché Redis. La aplicación completa se despliega con un único comando `docker compose up`, integrando los servicios de backend, base de datos, caché y frontend con verificación de salud (*healthcheck*).

1. Introducción

1.1. Contexto y motivación

Las bibliotecas institucionales ecuatorianas enfrentan un proceso de modernización tecnológica postergado: los registros manuales de préstamos generan aglomeraciones en ventanilla, el inventario es opaco para el lector y el cálculo de sanciones moratorias se realiza de forma manual y propensa a errores. Estudios recientes confirman que estas problemáticas son comunes en el sector bibliotecario latinoamericano [1, 2]. La Entrega 1A definió al SGB-SaaS como una plataforma 100 % web que centraliza estos procesos, automatiza el registro de inventario mediante la API de Google Books y ofrece un asistente conversacional basado en IA generativa. La presente entrega, Entrega 1B, constituye el primer módulo de código funcional del proyecto.

1.2. Objetivos de la entrega

1. Implementar un módulo de autenticación stateless con Spring Security 7 y JWT (HS256), incluyendo registro, login, logout con lista negra en Redis y renovación de token.
2. Configurar el acceso a datos de la entidad **Libro** mediante Spring Data JPA y Hibernate sobre PostgreSQL 16, con migraciones versionadas en Flyway.

3. Demostrar el CRUD completo de **Libro** (creación, lectura paginada, actualización y baja lógica) consumido desde un cliente Angular protegido por interceptor JWT.
4. Validar mediante pruebas automatizadas (JUnit 5) y pruebas de integración (Postman) el comportamiento de los flujos de autenticación y acceso a datos, documentando los resultados con métricas de rendimiento.

1.3. Alcance

En esta entrega se implementan: (1) el módulo de autenticación JWT completo (registro, login, logout, refresh); (2) el CRUD de la entidad **Libro** con sus tablas maestras de apoyo (**editoriales**, **idiomas**, **estados_libro**); (3) la infraestructura Docker Compose con PostgreSQL, Redis, backend y frontend. Quedan fuera del alcance de esta entrega: el módulo de préstamos y multas (**prestamos**, **multas**), las reservaciones, el chatbot con Gemini 2.0 Flash y los reportes estadísticos del Gerente, los cuales corresponden al cronograma de la Entrega 2 (Semana 12 en adelante), conforme a lo planificado en la Entrega 1A.

1.4. Estructura del documento

La sección 2 presenta el marco teórico que sustenta las decisiones de arquitectura. La sección 3 detalla el diseño del sistema: actualización del modelo C4, diagrama de clases UML, mapeo objeto-relacional, modelo entidad-relación y diagrama de secuencia del flujo JWT, junto con el script Flyway. La sección 4 describe la implementación: estructura del repositorio, configuración de Spring Boot, filtro JWT, endpoints e interceptor Angular. La sección 5 documenta las pruebas automatizadas. La sección 6 reporta las métricas de rendimiento con y sin caché Redis. La sección 7 detalla los controles de seguridad OWASP. La sección 8 describe el despliegue con Docker Compose. La sección 9 documenta el historial de Git. Finalmente, las secciones 10 y 11 presentan las conclusiones y la bibliografía.

2. Marco teórico

2.1. Autenticación *stateless* vs. *stateful*

En una arquitectura *stateful*, el servidor mantiene una sesión en memoria o en una base de datos para cada usuario autenticado, identificada mediante una cookie de sesión. En contraste, una arquitectura *stateless* no conserva información de sesión en el servidor: cada solicitud porta toda la información necesaria para autenticarla, típicamente mediante un JSON Web Token (JWT) firmado. El estándar JWT está definido en el RFC 7519 [6] y constituye el mecanismo de portador (*bearer token*) más utilizado en arquitecturas SPA/API, complementando al protocolo de autorización OAuth 2.0 (RFC 6749) [7]. Para aplicaciones de página única (SPA) como la del SGB-SaaS, el enfoque *stateless* permite escalar horizontalmente el backend sin sincronizar sesiones entre instancias, a costa de requerir mecanismos adicionales (como la lista negra en Redis) para la invalidación temprana de tokens.

2.2. Spring Security 7 y el modelo de filtros

Spring Security 7 organiza la autenticación y autorización mediante una cadena de filtros (*SecurityFilterChain*) que intercepta cada solicitud HTTP antes de que llegue al controlador [5, 10]. El *AuthenticationManager* delega en uno o más *AuthenticationProvider* la verificación de credenciales; en el caso del SGB-SaaS, un *DaoAuthenticationProvider* consulta un *UserDetailsService* personalizado (*UserDetailsServiceImpl*), que carga al usuario desde *UsuarioRepository* y compara la contraseña mediante *PasswordEncoder.matches*. Un filtro personalizado, *JwtAuthFilter*, se inserta antes del *UsernamePasswordAuthenticationFilter* para validar el token Bearer y poblar el *SecurityContextHolder* sin necesidad de sesión HTTP.

2.3. ORM y mapeo objeto-relacional

El uso de un *Object-Relational Mapping* (ORM) como Hibernate, a través de la abstracción Spring Data JPA, elimina la necesidad de escribir SQL manual para operaciones CRUD básicas y reduce drásticamente el riesgo de inyección SQL frente al uso directo de JDBC [4]. Spring Data JPA permite declarar repositorios como interfaces (*JpaRepository<Libro, Long>*) que generan automáticamente consultas a partir del nombre del método (*derived queries*), además de soportar paginación y ordenamiento de forma nativa. La documentación oficial de Hibernate describe el ciclo de vida de las entidades (*transient*, *persistent*, *detached*) y el caché de primer nivel asociado al *EntityManager*, aspectos que el equipo consideró al definir la propiedad `spring.jpa.open-in-view: false` para evitar excepciones de tipo *LazyInitializationException* fuera del contexto transaccional.

2.4. Migraciones de base de datos con Flyway

Flyway gestiona el versionado del esquema de base de datos mediante archivos SQL nombrados con un prefijo de versión: los archivos *V1__*, *V2__*, etc. son migraciones *versionadas*, que se ejecutan una única vez y en orden estricto, mientras que los archivos *R__* son migraciones *repetibles*, que se vuelven a ejecutar cada vez que su contenido cambia (típicamente para vistas o funciones) [12]. Esta convención garantiza que los entornos de desarrollo, pruebas y producción converjan al mismo esquema, eliminando discrepancias manuales entre bases de datos del equipo.

2.5. OWASP y seguridad de JWT

El OWASP Top 10:2021 identifica en la categoría A07 (*Identification and Authentication Failures*) los riesgos asociados a una gestión deficiente de credenciales y tokens [8]. Para mitigarlos, el SGB-SaaS aplica: (1) hashing de contraseñas con BCrypt costo 12; (2) tokens de acceso de corta duración (1 hora) acompañados de un *refresh token* de mayor duración (7 días) almacenado en una cookie *HttpOnly*; y (3) almacenamiento del *accessToken* en memoria del cliente Angular, evitando *localStorage* por su exposición a ataques de Cross-Site Scripting (XSS). Adicionalmente, dado que JWT es por definición *stateless*, se incorpora Redis [11] como lista negra de identificadores de token (*ITI*) revocados, permitiendo invalidar sesiones antes de su expiración natural.

3. Diseño del sistema

3.1. Actualización del modelo C4 – Nivel 2 (Contenedores)

Respecto al diagrama C4 Nivel 2 presentado en la Entrega 1A (Figura 2) [14], esta entrega materializa los contenedores **SPA Angular** (TypeScript, puerto 4200) y **Backend Spring Boot** (Java 21, puerto 8080), e incorpora el contenedor **Redis** (puerto 6379) como caché y almacén de la lista negra JWT, no contemplado explícitamente en el diagrama original. La comunicación Angular–Spring Boot se realiza vía HTTPS/JSON con el token JWT en el encabezado `Authorization`; la comunicación Spring Boot–PostgreSQL vía JDBC/SQL a través de Hibernate; y la comunicación Spring Boot–Redis vía el protocolo RESP (cliente Lettuce, incluido en `spring-boot-starter-data-redis`).

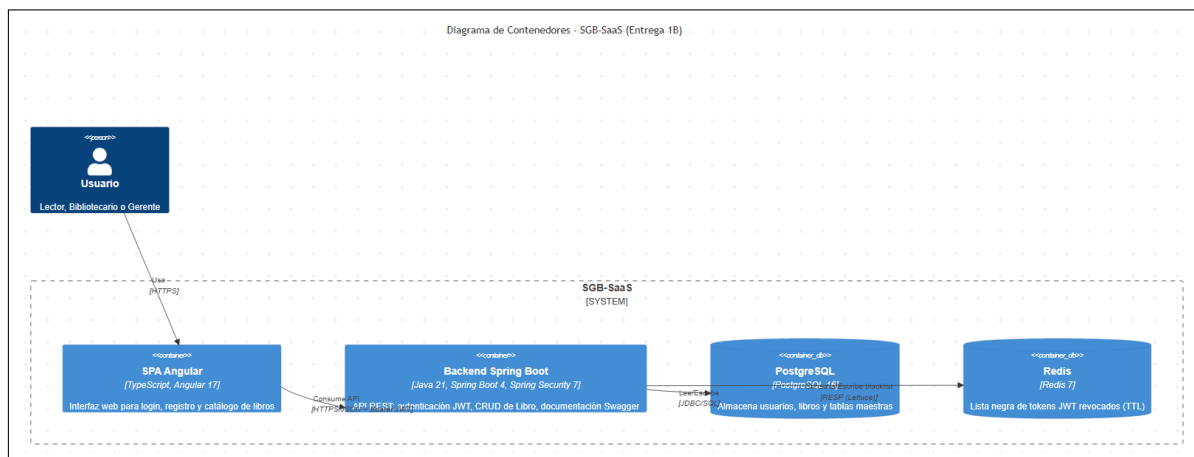


Figura 1: Diagrama C4 Nivel 2 actualizado. Contenedores: SPA Angular (TypeScript, puerto 4200), Backend Spring Boot (Java 21, puerto 8080, con Spring Security + JPA + springdoc-openapi), PostgreSQL 16 (puerto 5432) y Redis 7 (puerto 6379, caché y blacklist JWT). Flechas: HTTPS/JSON entre Angular y Spring Boot; JDBC/SQL entre Spring Boot y PostgreSQL; RESP entre Spring Boot y Redis.

3.2. Diagrama de clases UML refactorizado

El diagrama de clases representa la estructura del backend Spring Boot organizada en los paquetes `entity`, `dto`, `service`, `repository`, `controller`, `security` y `config`. A continuación se describe el contenido de cada paquete implementado en esta entrega:

- `entity:Usuario` (- `id`: Long, - `nombre`: String, - `correo`: String, - `passwordHash`: String, - `rol`: Rol, - `activo`: boolean, - `creadoEn`: Instant); `Libro` (- `id`: Long, - `isbn`: String, - `titulo`: String, - `resumen`: String, - `portadaUrl`: String, - `anioPublicacion`: Short, - `stockTotal`: Short, - `stockDisponible`: Short, - `activo`: boolean); `Editorial`, `Idioma`, `EstadoLibro` (tablas maestras, relación @ManyToMany desde Libro).
- `dto:LoginRequestDTO`, `RegistroRequestDTO`, `TokenResponseDTO` (`accessToken`, `refreshToken`, `expiresIn`); `LibroRequestDTO`, `LibroResponseDTO` – todos como record, sin anotaciones @Entity.

- repository: `UsuarioRepository` extends `JpaRepository<Usuario, Long>` con `findByCorreo(String correo): Optional<Usuario>`; `LibroRepository` extends `JpaRepository<Libro, Long>` con `findByIsbn(String isbn): Optional<Libro>` y `findByActivoTrue(Pageable pageable): Page<Libro>`.
- service: `AuthService` (campos final: `UsuarioRepository`, `PasswordEncoder`, `JwtService`, `AuthenticationManager`; métodos `registrar()`, `login()`, `logout()`, `refresh()`); `LibroService` (campo final `LibroRepository`; métodos `listar(Pageable): Page<LibroResponseDTO>`, `buscarPorId(Long): LibroResponseDTO`, `crear()`, `actualizar()`, `eliminar()` – soft delete).
- controller: `AuthController (/api/auth/*)`; `LibroController (/api/v1/libros)`.
- security: `JwtService` (genera/valida tokens, depende de `jjwt`); `JwtAuthFilter` extends `OncePerRequestFilter` (depende de `JwtService` y `RedisTemplate`); `UserDetailsServiceImpl` implements `UserDetailsService` (depende de `UsuarioRepository`).
- config: `SecurityConfig` (define `SecurityFilterChain`, habilita `@EnableMethodSecurity`, configura `CORS` y cabeceras de seguridad `HTTP`; ubicado en paquete `config`); `RedisConfig` (configura `RedisTemplate<String, String>` con `GenericJackson2JsonRedisSerializer` para caché y blacklist `JWT`).

Relaciones principales: `LibroController` → `LibroService` → `LibroRepository` (asociación); `LibroService` 99K `LibroResponseDTO` (dependencia, mapeo); `UserDetailsServiceImpl` implements `UserDetailsService`; `JwtAuthFilter` extends `OncePerRequestFilter`; `Libro` → `Editorial`, `Libro` → `Idioma`, `Libro` → `EstadoLibro` (asociación `@ManyToOne`).

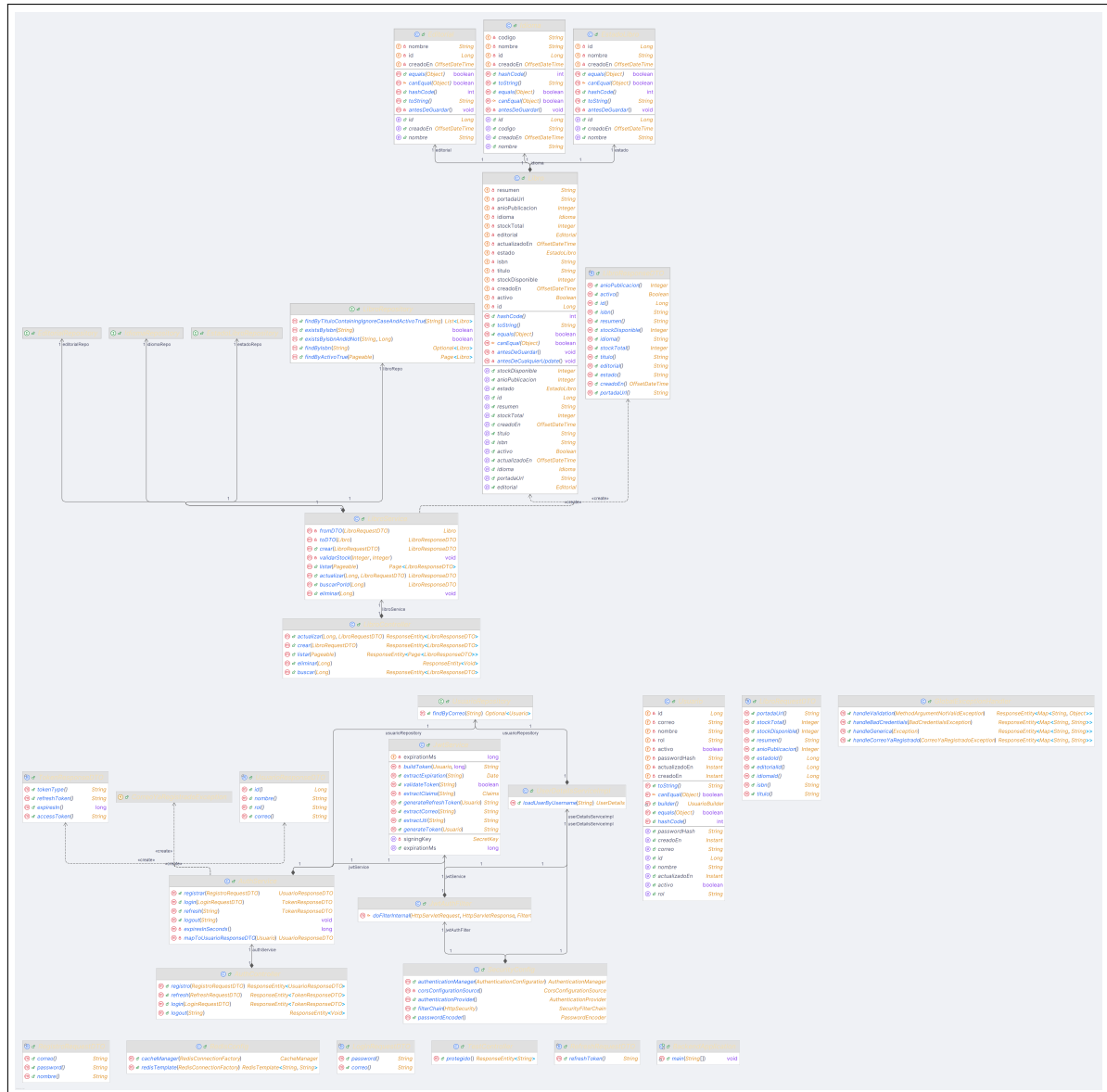


Figura 2: Diagrama de clases UML refactorizado de la Entrega 1B, exportado desde IntelliJ IDEA (Diagrams > Show Diagram), agrupado por paquetes **entity**, **dto**, **service**, **repository**, **controller**, **security** y **config**, mostrando atributos tipados, visibilidad y relaciones de asociación, dependencia e implementación.

3.3. Tabla de correspondencia Java ↔ PostgreSQL

Atributo Java	Tipo Java	Tipo PostgreSQL	Restricciones y justificación
id	Long	BIGSERIAL	PK; secuencia PostgreSQL; mejor rendimiento en <i>joins</i> que UUID.
isbn	String	VARCHAR(13)	UNIQUE NOT NULL; índice CREATE UNIQUE INDEX.
titulo	String	VARCHAR(255)	NOT NULL.

Atributo Java	Tipo Java	Tipo PostgreSQL	Restricciones y justificación
resumen	String	TEXT	Nulo; proviene de Google Books API (fuera de alcance en 1B).
portadaUrl	String	VARCHAR(1000)	Nulo; URL de portada.
anioPublicacion	Short	SMALLINT	CHECK (1000–2100).
editorialId	Integer (@ManyToOne)	INTEGER	FK → editoriales.id, ON DELETE RESTRICT.
idiomaId	Integer (@ManyToOne)	INTEGER	FK → idiomas.id, ON DELETE RESTRICT.
estadoId	Integer (@ManyToOne)	INTEGER	FK → estados_libro.id.
stockTotal	Short	SMALLINT	DEFAULT 1, CHECK (≥ 0).
stockDisponible	Short	SMALLINT	DEFAULT 1, CHECK (≥ 0 y $\leq \text{stock_total}$).
activo	boolean	BOOLEAN	DEFAULT TRUE; soft delete.
creadoEn / actualizadoEn	OffsetDateTime	TIMESTAMPTZ	UTC; gestionados por @PrePersist y @PreUpdate; trigger set_actualizado_en como respaldo a nivel BD.
<i>Entidad Usuario (módulo de autenticación)</i>			
id	Long	BIGSERIAL	PK; secuencia PostgreSQL.
correo	String	VARCHAR(150)	UNIQUE NOT NULL; CREATE UNIQUE INDEX.
passwordHash	String	VARCHAR(255)	BCrypt costo 12; nunca serializado (@JsonIgnore).
rol	enum Rol	VARCHAR(20)	CHECK: ROLE_LECTOR, ROLE_BIBLIOTECARIO, ROLE_GERENTE; sin ENUM nativo PostgreSQL.
activo	boolean	BOOLEAN	DEFAULT TRUE; soft delete / bloqueo.
creadoEn / actualizadoEn	Instant	TIMESTAMPTZ	UTC; asignados manualmente en AuthService (Instant.now()); trigger set_actualizado_en actualiza actualizado_en en BD.

Cuadro 1: Tabla de correspondencia Java ↔ PostgreSQL para las entidades Libro y Usuario implementadas en esta entrega.

3.4. Diagrama Entidad-Relación (DER) y diccionario de datos

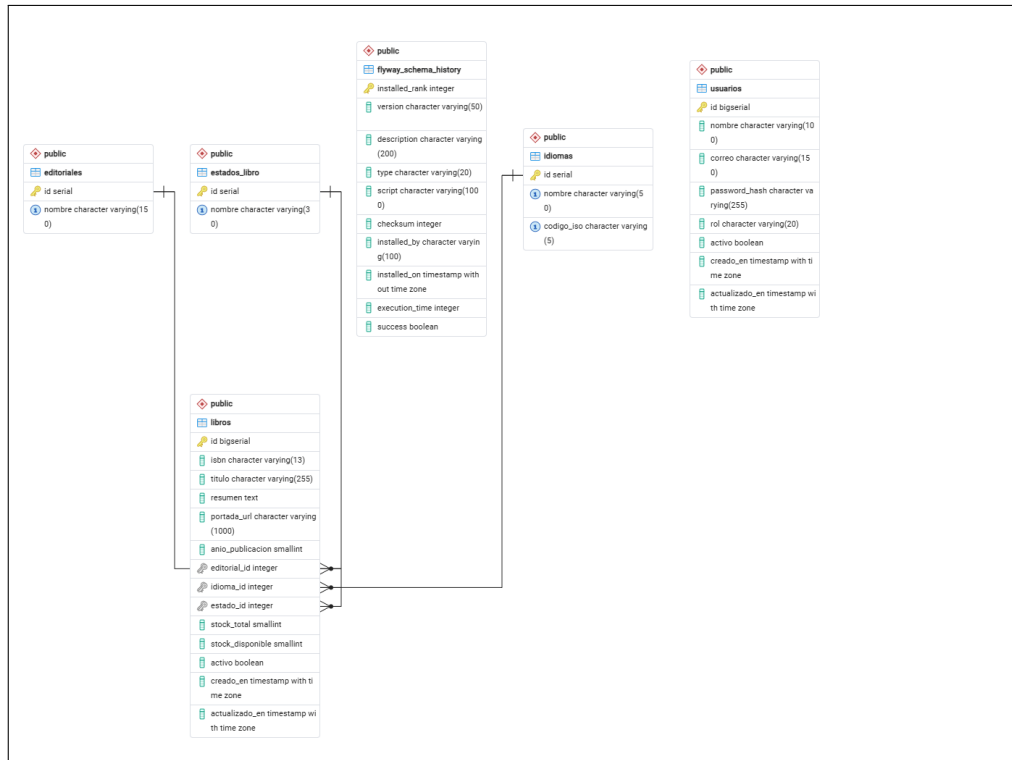


Figura 3: Diagrama Entidad-Relación de las tablas implementadas en la Entrega 1B (usuarios, libros, editoriales, idiomas, estados_libro), generado desde pgAdmin 4 (Tools >ERD Tool), mostrando claves primarias, foráneas, tipos de dato PostgreSQL y cardinalidades.

Columna	Tipo	Nulo	Default	Descripción
Tabla: usuarios				
id	BIGSERIAL	No	auto	Identificador único.
nombre	VARCHAR(100)	No	—	Nombre del usuario.
correo	VARCHAR(150)	No	—	Correo de acceso (login).
password_hash	VARCHAR(255)	No	—	Hash BCrypt de la contraseña.
rol	VARCHAR(20)	No	ROLE_LECTOR	Rol Spring Security.
activo	BOOLEAN	No	TRUE	Soft delete / bloqueo de cuenta.
creado_en	TIMESTAMP	No	NOW()	Fecha de alta.
actualizado_en	TIMESTAMP	No	NOW()	Última modificación (trigger).
Tabla: libros				
id	BIGSERIAL	No	auto	Identificador del título.
isbn	VARCHAR(13)	No	—	Código ISBN-13, único.
titulo	VARCHAR(255)	No	—	Título completo.
resumen	TEXT	Sí	NULL	Sinopsis.
portada_url	VARCHAR(1000)	Sí	NULL	URL de portada.
anio_publicacion	SMALLINT	No	—	CHECK 1000–2100.
editorial_id	INTEGER	No	—	FK → editoriales.id.
idioma_id	INTEGER	No	—	FK → idiomas.id.
estado_id	INTEGER	No	—	FK → estados_libro.id.
stock_total	SMALLINT	No	1	Ejemplares totales.
stock_disponible	SMALLINT	No	1	Ejemplares en estante.
activo	BOOLEAN	No	TRUE	Soft delete.
creado_en	TIMESTAMP	No	NOW()	Auditoría temporal.
/ actualizado_en				

Cuadro 2: Diccionario de datos de las tablas `usuarios` y `libros`. Las tablas maestras `editoriales`, `idiomas` y `estados_libro` siguen la estructura (`id` SERIAL, `nombre` VARCHAR UNIQUE NOT NULL). El diccionario completo de las 24 tablas del sistema se mantiene en `docs/diccionario_datos.md`.

3.5. Diagrama de secuencia: flujo de autenticación JWT

El flujo de autenticación implementado sigue los 11 pasos definidos en la guía de la entrega:

1. El cliente Angular envía `POST /api/auth/login` con `{correo, password}`.
2. `AuthController` delega en `AuthenticationManager.authenticate()`.
3. Spring Security consulta `UserDetailsService.loadUserByUsername()`.
4. `UserDetailsServiceImpl` consulta `UsuarioRepository.findByCorreo()`.
5. PostgreSQL devuelve el usuario; Hibernate lo mapea a la entidad `Usuario`.

6. Spring Security verifica el hash BCrypt con `PasswordEncoder.matches()`.
7. `JwtService.generateToken()` firma el JWT (HS256) con la clave secreta (`JWT_SECRET`).
8. El servidor devuelve `{accessToken, refreshToken, expiresIn}`.
9. Angular guarda el `accessToken` en memoria (servicio `AuthService`) y el `refreshToken` en una cookie `HttpOnly`.
10. En cada solicitud posterior, `JwtInterceptor` agrega el encabezado `Authorization: Bearer [token]`.
11. `JwtAuthFilter` valida el token, consulta Redis (blacklist de JTI) y, si es válido, establece el `SecurityContext`.

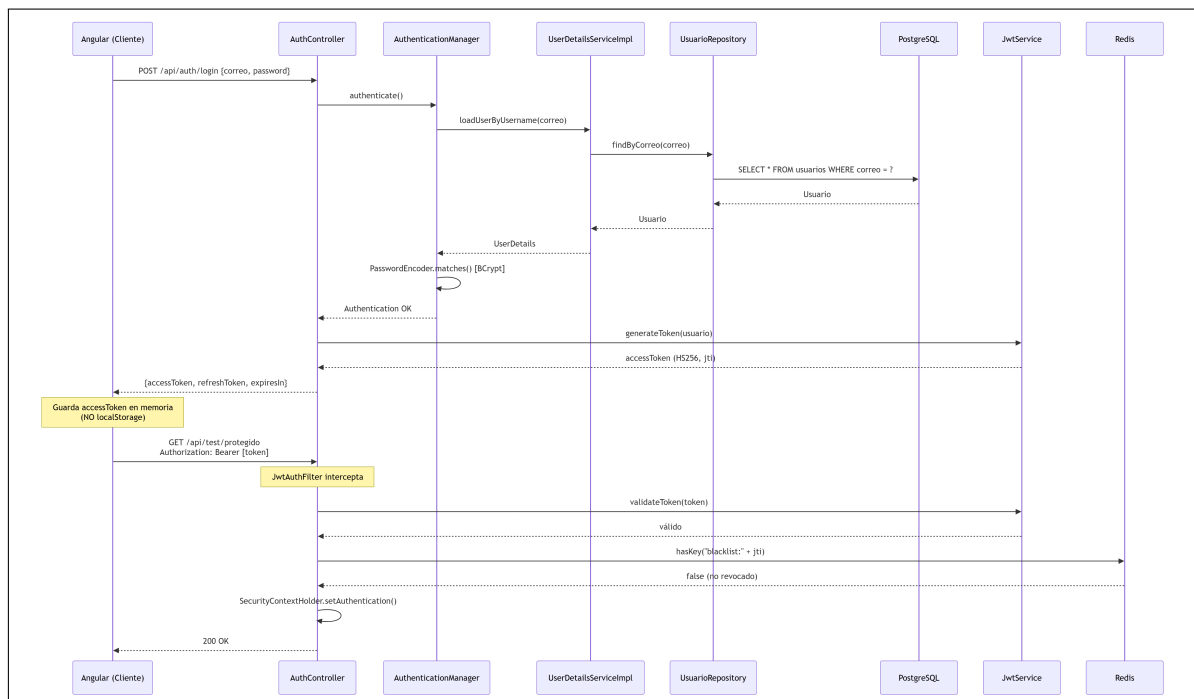


Figura 4: Diagrama de secuencia UML del flujo de autenticación JWT (11 pasos), desde la solicitud de login en Angular hasta el establecimiento del `SecurityContext` en el filtro `JwtAuthFilter`, incluyendo la consulta a Redis para verificación de blacklist.

3.6. Script Flyway de migración inicial

Listing 1: database/migrations/V1__schema_inicial.sql

```

1 -- V1__schema_inicial.sql
2 -- Flyway ejecuta este archivo en orden; nunca modificar
3 -- un archivo V ya ejecutado: crear V2__ en su lugar.
4
5 CREATE EXTENSION IF NOT EXISTS "uuid-oss";
6
7 -- ===== TABLAS MAESTRAS =====
8 CREATE TABLE editoriales (
9     id SERIAL PRIMARY KEY,
10    nombre VARCHAR(150) NOT NULL UNIQUE
11 );

```

```

12
13 CREATE TABLE idiomas (
14     id SERIAL PRIMARY KEY,
15     nombre VARCHAR(50) NOT NULL UNIQUE,
16     codigo_iso VARCHAR(5) NOT NULL UNIQUE
17 );
18
19 CREATE TABLE estados_libro (
20     id SERIAL PRIMARY KEY,
21     nombre VARCHAR(30) NOT NULL UNIQUE
22     -- Valores: 'ACTIVO', 'DADO_DE_BAJA', 'EN_REPARACION', 'PERDIDO'
23 );
24
25 -- ===== TABLA DE USUARIOS =====
26 CREATE TABLE usuarios (
27     id BIGSERIAL PRIMARY KEY,
28     nombre VARCHAR(100) NOT NULL,
29     correo VARCHAR(150) NOT NULL,
30     password_hash VARCHAR(255) NOT NULL,
31     rol VARCHAR(20) NOT NULL DEFAULT 'ROLE_LECTOR',
32     activo BOOLEAN NOT NULL DEFAULT TRUE,
33     creado_en TIMESTAMPTZ NOT NULL DEFAULT NOW(),
34     actualizado_en TIMESTAMPTZ NOT NULL DEFAULT NOW()
35 );
36
37 CREATE UNIQUE INDEX idx_usuarios_correo ON usuarios (correo);
38
39 ALTER TABLE usuarios
40     ADD CONSTRAINT chk_rol
41     CHECK (rol IN ('ROLE_LECTOR', 'ROLE_BIBLIOTECARIO', 'ROLE_GERENTE'));
42
43 -- ===== TABLA DE LIBROS =====
44 CREATE TABLE libros (
45     id BIGSERIAL PRIMARY KEY,
46     isbn VARCHAR(13) NOT NULL,
47     titulo VARCHAR(255) NOT NULL,
48     resumen TEXT,
49     portada_url VARCHAR(1000),
50     anio_publicacion SMALLINT NOT NULL,
51     editorial_id INTEGER NOT NULL REFERENCES editoriales(id) ON DELETE
    RESTRICT,
52     idioma_id INTEGER NOT NULL REFERENCES idiomas(id) ON DELETE
    RESTRICT,
53     estado_id INTEGER NOT NULL REFERENCES estados_libro(id) ON DELETE
    RESTRICT,
54     stock_total SMALLINT NOT NULL DEFAULT 1,
55     stock_disponible SMALLINT NOT NULL DEFAULT 1,
56     activo BOOLEAN NOT NULL DEFAULT TRUE,
57     creado_en TIMESTAMPTZ NOT NULL DEFAULT NOW(),
58     actualizado_en TIMESTAMPTZ NOT NULL DEFAULT NOW(),
59     CONSTRAINT chk_anio_publicacion CHECK (anio_publicacion BETWEEN 1000 AND 2100),
60     CONSTRAINT chk_stock_total CHECK (stock_total >= 0),
61     CONSTRAINT chk_stock_disponible CHECK (stock_disponible >= 0 AND
    stock_disponible <= stock_total)
62 );
63
64 CREATE UNIQUE INDEX idx_libros_isbn ON libros (isbn);
65

```

```

66 -- ===== TRIGGER PARA actualizado_en =====
67 CREATE OR REPLACE FUNCTION set_actualizado_en()
68 RETURNS TRIGGER AS $$
69 BEGIN
70     NEW.actualizado_en = NOW();
71     RETURN NEW;
72 END;
73 $$ LANGUAGE plpgsql;
74
75 CREATE TRIGGER trg_usuarios_actualizado_en
76 BEFORE UPDATE ON usuarios
77 FOR EACH ROW EXECUTE FUNCTION set_actualizado_en();
78
79 CREATE TRIGGER trg_libros_actualizado_en
80 BEFORE UPDATE ON libros
81 FOR EACH ROW EXECUTE FUNCTION set_actualizado_en();
82
83 -- [Las tablas de prestamos, multas, reservaciones, bitacora_auditoria
84 -- y el resto del esquema de 24 tablas se incorporaran en V2_ y
85 -- posteriores, conforme al cronograma de la Entrega 2]

```

4. Implementación

4.1. Estructura del repositorio monorepo

Ruta	Contenido
backend-springboot/src/main/java/	Código fuente Java organizado en paquetes <code>entity</code> , <code>dto</code> , <code>service</code> , <code>repository</code> , <code>controller</code> , <code>security</code> , <code>config</code> .
backend-springboot/src/main/resources/	<code>application.yml</code> y migraciones Flyway en <code>db/migration/</code> .
backend-springboot/src/test/	Pruebas unitarias JUnit 5 + Mockito puras: <code>BackendApplicationTests</code> (carga de contexto con <code>@MockitoBean</code>), <code>AuthServiceTest</code> (5 pruebas del módulo de autenticación), <code>LibroServiceTest</code> (5 pruebas del CRUD de Libro).
frontend-angular/src/app/	Módulos Angular: <code>auth/</code> (login, registro), <code>core/</code> (interceptor JWT, guard), <code>libros/</code> (CRUD con paginación).
database/migrations/	Scripts Flyway <code>V1_</code> , <code>V2_</code> , etc.
docker-compose.yml	Servicios: <code>backend</code> , <code>postgres</code> , <code>redis</code> , <code>frontend</code> (Nginx).
.env.example	Plantilla de variables: <code>JWT_SECRET</code> , <code>DB_URL</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code> , <code>REDIS_HOST</code> , <code>REDIS_PORT</code> .
docs/adr/ADR-003-jwt-redis.md	Justificación de Redis para blacklist JWT.
.github/workflows/ci.yml	Pipeline GitHub Actions: compila y ejecuta los 11 tests (<code>mvnw clean verify</code>) en cada push a <code>develop</code> y <code>main</code> .

Cuadro 3: Estructura real del repositorio monorepo al momento de la Entrega 1B.

4.2. Configuración de Spring Boot

Listing 2: backend-springboot/src/main/resources/application.yml

```

1 spring:
2   datasource:
3     url: ${DB_URL:jdbc:postgresql://localhost:5432/sgb_db}
4     username: ${DB_USER:sgb_user}
5     password: ${DB_PASSWORD:changeme}
6     driver-class-name: org.postgresql.Driver
7   jpa:
8     hibernate:
9       ddl-auto: validate      # Flyway gestiona el DDL
10      show-sql: false         # true solo en desarrollo local
11      open-in-view: false     # evitar LazyInitializationException
12      properties:
13        hibernate:
14          format_sql: true
15          default_schema: public
16      flyway:
17        enabled: true
18        locations: classpath:db/migration
19        baseline-on-migrate: true
20      data:
21        redis:
22          host: ${REDIS_HOST:localhost}
23          port: ${REDIS_PORT:6379}
24
25 security:
26   jwt:
27     secret: ${JWT_SECRET}           # min. 256 bits, generado con openssl rand
28     -hex 32
29     expiration-ms: 3600000          # 1 hora
30     refresh-expiration-ms: 604800000 # 7 dias
31
32 springdoc:
33   api-docs:
34     path: /api/docs
35   swagger-ui:
36     path: /swagger-ui.html
37
38 management:
39   endpoints:
40     web:
41     exposure:
42       include: health, info, metrics

```

4.3. Implementación del filtro JWT

`JwtAuthFilter` extiende `OncePerRequestFilter` (garantiza una única ejecución por solicitud) y se registra en `SecurityConfig` antes de `UsernamePasswordAuthenticationFilter`. Su lógica:

1. Extrae el token del encabezado `Authorization: Bearer [token]`; si no existe o no tiene el prefijo esperado, continúa la cadena sin autenticar.
2. Llama a `JwtService.validateToken(token)`, que verifica la firma HS256 y la

expiración (`exp`).

- Obtiene el `jti` (claim *JWT ID*) del token y consulta `RedisTemplate.hasKey("blacklist:" + jti)`; si existe, rechaza la solicitud con 401.
- Si el token es válido y no está en la blacklist, construye un `UsernamePasswordAuthenticationToken` con el usuario y rol extraídos del claim `rol`, y lo establece en `SecurityContextHolder.getContext()`.

Listing 3: Esqueleto de `JwtAuthFilter` (paquete `security`)

```

1 @Component
2 @RequiredArgsConstructor
3 public class JwtAuthFilter extends OncePerRequestFilter {
4
5     private final JwtService jwtService;
6     private final RedisTemplate<String, String> redisTemplate;
7     private final UserDetailsServiceImpl userDetailsService;
8
9     @Override
10    protected void doFilterInternal(HttpServletRequest request,
11                                    HttpServletResponse response,
12                                    FilterChain filterChain)
13        throws java.io.IOException, jakarta.servlet.ServletException {
14
15        String header = request.getHeader("Authorization");
16        if (header == null || !header.startsWith("Bearer ")) {
17            filterChain.doFilter(request, response);
18            return;
19        }
20
21        String token = header.substring(7);
22
23        if (!jwtService.validateToken(token)) {
24            filterChain.doFilter(request, response);
25            return;
26        }
27
28        String jti = jwtService.extractJti(token);
29        Boolean enBlacklist = redisTemplate.hasKey("blacklist:" + jti);
30        if (Boolean.TRUE.equals(enBlacklist)) {
31            filterChain.doFilter(request, response);
32            return;
33        }
34
35        String correo = jwtService.extractCorreo(token);
36        var userDetails = userDetailsService.loadUserByUsername(correo);
37
38        var authToken = new UsernamePasswordAuthenticationToken(
39            userDetails, null, userDetails.getAuthorities());
40        SecurityContextHolder.getContext().setAuthentication(authToken);
41
42        filterChain.doFilter(request, response);
43    }
44 }

```

4.4. Endpoints implementados

Método	URL	Auth	Descripción
POST	/api/auth/registro	No	Registrar nuevo usuario; devuelve el usuario creado (sin hash).
POST	/api/auth/login	No	Autenticar; devuelve <code>accessToken</code> y <code>refreshToken</code> .
POST	/api/auth/logout	JWT	Agrega el JTI a la blacklist de Redis.
POST	/api/auth/refresh	RefreshToken	Emite un nuevo <code>accessToken</code> sin re-autenticar.
GET	/api/test/protegido	JWT	Endpoint de prueba para validar el flujo de autenticación (acceso autorizado/denegado).
GET	/api/v1/libros	JWT	Listar con paginación: <code>?page=0&size=10&sort=id,asc</code> .
GET	/api/v1/libros/{id}	JWT	Buscar por ID; 404 si no existe.
POST	/api/v1/libros	JWT	Crear; valida con <code>@Valid</code> ; 201 Created.
PUT	/api/v1/libros/{id}	JWT	Actualizar completo; 200 OK.
DELETE	/api/v1/libros/{id}	JWT	Soft delete (<code>activo=false</code>); 204 No Content.
GET	/api/docs	No	Especificación OpenAPI en JSON/YAML (importable en Postman o herramientas de API).
GET	/swagger-ui.html	No	Swagger UI — interfaz web de documentación de la API generada por Springdoc OpenAPI.

Cuadro 4: Endpoints implementados en la Entrega 1B.

4.5. Estructura del JWT emitido

Claim	Valor de ejemplo	Descripción
sub	"17"	ID del usuario (sujeto del token).
correo	"lector@uteq.edu.ec"	Correo del usuario.
rol	ROLE_LECTOR"	Rol Spring Security.
iat	1749427200	<i>Issued at</i> (timestamp Unix).
exp	1749430800	<i>Expiration</i> (<code>iat</code> + 3600 s).
jti	..a1b2c3d4-e5f6-..."	ID único del JWT; usado para blacklist en Redis.

Cuadro 5: Payload decodificado del `accessToken` emitido por el sistema (Base64, header omitido).

4.6. Implementación Angular: interceptor JWT

El `accessToken` se almacena exclusivamente en una variable privada del servicio `AuthService` (en memoria, nunca en `localStorage`). El `JwtInterceptor` (`HttpInterceptorFn`) clona cada solicitud saliente y, si existe un token en memoria, agrega el encabezado `Authorization: Bearer [token]`. Para evitar un bucle infinito de redirección ante errores 401, el manejo automático de logout se aplica únicamente a endpoints de negocio: si la solicitud fallida apunta a una ruta de autenticación (`/auth/*`), el error se propaga sin invocar al servicio de logout. El `AuthGuard` (`CanActivateFn`) verifica `authService.isLoggedIn()` antes de activar rutas protegidas y, si no hay sesión activa, redirige al usuario a `/login`.

Listing 4: `core/interceptors/jwt.interceptor.ts`

```
1 export const jwtInterceptor: HttpInterceptorFn = (req, next) => {
2   const authService = inject(AuthService);
3   const router = inject(Router);
4   const token = authService.getAccessToken();
5
6   const authReq = token
7     ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
8     : req;
9
10  return next(authReq).pipe(
11    catchError((error) => {
12      // Excluir rutas /auth/* para evitar loop infinito ante 401
13      const isAuthEndpoint = req.url.includes('/auth/');
14      if (error.status === 401 && !isAuthEndpoint) {
15        authService.logout();
16        router.navigate(['/login']);
17      }
18      return throwError(() => error);
19    })
20  );
21 };
```

5. Pruebas automatizadas

5.1. Pruebas unitarias del servicio de autenticación

#	Caso de prueba	Método de prueba	Esperado	Result.
<i>AuthServiceTest — autenticación (JUnit 5 + Mockito)</i>				
1	Login con credenciales correctas	<code>loginExitoso()</code>	<code>TokenResponseDTO</code> con tokens no vacíos	PASS
2	Login con contraseña incorrecta	<code>loginClaveIncorrecta()</code>	<code>BadCredentialsException</code>	PASS
3	Registro con correo ya existente	<code>registroCorreo-Duplicado()</code>	<code>CorreoYaRegistradoException</code> ; <code>save()</code> no invocado	PASS
4	Logout guarda JTI en blacklist Redis	<code>logoutGuarda-TokenEnBlacklist()</code>	<code>valueOperations.set</code> con clave <code>"blacklist:jti"</code> y TTL	PASS
5	Refresh con token válido retorna nuevo <code>accessToken</code>	<code>refreshConToken-Valido()</code>	Nuevo <code>accessToken</code> ; mismo <code>refreshToken</code>	PASS
<i>LibroServiceTest — CRUD de Libro (JUnit 5 + Mockito)</i>				
6	Crear libro con ISBN nuevo retorna DTO	<code>crearLibro_cuando-IsbnNuevo_retornaDTO()</code>	DTO con ISBN y título correctos	PASS
7	Crear libro con ISBN duplicado lanza excepción	<code>crearLibro_cuando-IsbnDuplicado_lanza-Excepcion()</code>	<code>IllegalArgumentException</code> "ISBN ya registrado"	PASS
8	Buscar libro inexistente lanza 404	<code>buscarPorId_cuando-NoExiste_lanza-EntityNotFound()</code>	<code>EntityNotFoundException</code> con ID en mensaje	PASS
9	Eliminar libro activa soft delete	<code>eliminar_cuando-Existe_desactiva-Libro()</code>	<code>activo=false</code> ; <code>save()</code> invocado	PASS
10	Listar retorna página de resultados	<code>listar_retorna-PaginaDeLibros()</code>	Page con 1 elemento y título correcto	PASS
<i>BackendApplicationTests — integración de contexto (SpringBootTest)</i>				
11	Contexto Spring Boot carga sin errores	<code>contextLoads()</code>	Contexto inicia liza con mocks JPA/Redis/Flyway	PASS

Cuadro 6: Once pruebas automatizadas implementadas en la Entrega 1B: 5 unitarias de autenticación (`AuthServiceTest`), 5 unitarias de CRUD (`LibroServiceTest`) y 1 de integración de contexto (`BackendApplicationTests`).

```

11
12 :: Spring Boot ::                (v4.0.6)
13
14 2026-06-22T17:43:08.272-05:00 INFO 16240 --- [backend] [      main] c.uteq.backend.BackendApplicationTests : Starting Backer
15 2026-06-22T17:43:08.273-05:00 INFO 16240 --- [backend] [      main] c.uteq.backend.BackendApplicationTests : No active profi
16 Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please
17 WARNING: A Java agent has been loaded dynamically (C:\Users\marlo\.m2\repository\net\bytebuddy\byte-buddy-agent\1.17.8\byte-buddy-ag
18 WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
19 WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
20 WARNING: Dynamic loading of agents will be disallowed by default in a future release
21 OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
22 2026-06-22T17:43:11.210-05:00 INFO 16240 --- [backend] [      main] eAuthenticationProviderManagerConfigurer : Global Authent
23 2026-06-22T17:43:11.211-05:00 WARN 16240 --- [backend] [      main] r$InitializeUserDetailsManagerConfigurer : Global Authent
24 2026-06-22T17:43:12.882-05:00 INFO 16240 --- [backend] [      main] o.s.b.a.e.web.EndpointLinksResolver : Exposing 3 endp
25 2026-06-22T17:43:12.954-05:00 INFO 16240 --- [backend] [      main] c.uteq.backend.BackendApplicationTests : Started Backend
26 2026-06-22T17:43:13.030-05:00 WARN 16240 --- [backend] [      main] o.s.core.events.SpringDocAppInitializer : SpringDoc /api/
27 2026-06-22T17:43:13.031-05:00 WARN 16240 --- [backend] [      main] o.s.core.events.SpringDocAppInitializer : SpringDoc /swag
28 [INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.506 s -- in com.uteq.backend.BackendApplicationTests
29 [INFO] Running com.uteq.backend.service.AuthServiceTest
30 [INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.513 s -- in com.uteq.backend.service.AuthServiceTest
31 [INFO] Running com.uteq.backend.service.LibroServiceTest
32 [INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.096 s -- in com.uteq.backend.service.LibroServiceTest
33 [INFO]
34 [INFO] Results:
35 [INFO]
36 [INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
37 [INFO]
38 [INFO]
39 [INFO] --- jar:3.4.2:jar (default-jar) @ backend ---
40 [INFO] Building jar: D:\UTEQ\5to semestre\appWeb\b1 - puliendo repositorio\sgb-saas\backend-springboot\target\backend-0.0.1-SNAPSHOT
41 [INFO]
42 [INFO] --- spring-boot:4.0.6:repackage (repackage) @ backend ---
43 [INFO] Replacing main artifact D:\UTEQ\5to semestre\appWeb\b1 - puliendo repositorio\sgb-saas\backend-springboot\target\backend-0.0.1-SNAPSHOT
44 [INFO] The original artifact has been renamed to D:\UTEQ\5to semestre\appWeb\b1 - puliendo repositorio\sgb-saas\backend-springboot\t
45 [INFO] -----
46 [INFO] BUILD SUCCESS
47 [INFO] -----
48 [INFO] Total time: 13.485 s
49 [INFO] Finished at: 2026-06-22T17:43:14-05:00
50 [INFO] -----

```

Figura 5: Captura de la salida de `./mvnw test` mostrando las 11 pruebas en estado PASS (5 AuthServiceTest + 5 LibroServiceTest + 1 BackendApplicationTests).

5.2. Pruebas de integración con Postman

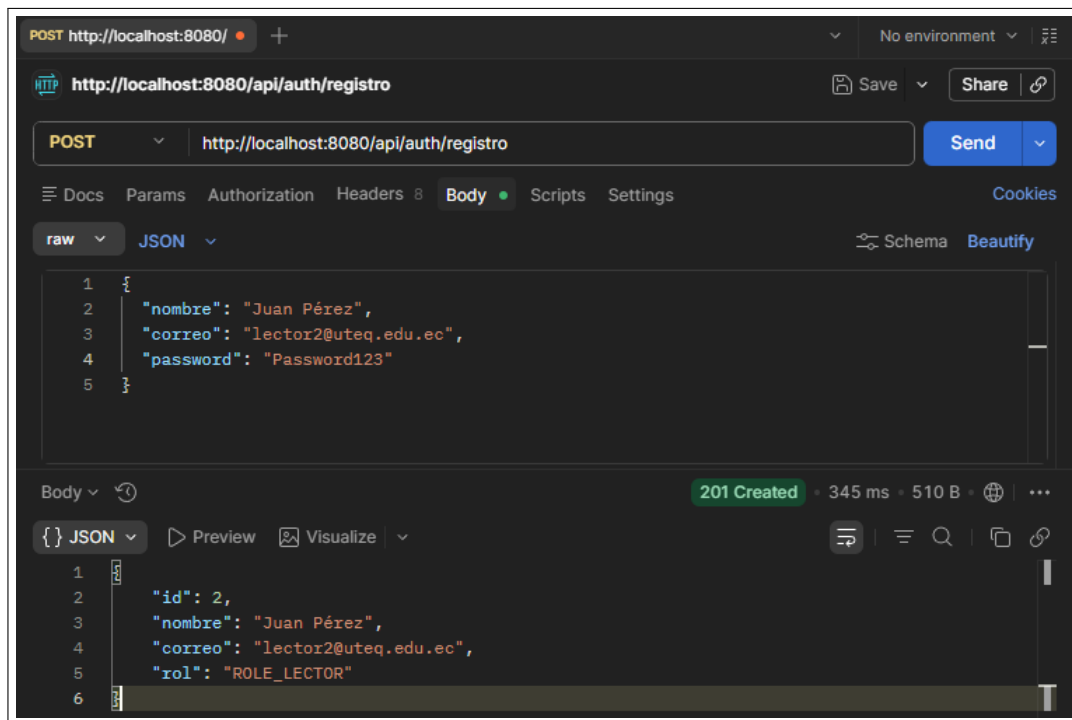


Figura 6: Postman: registro de un nuevo usuario (POST /api/auth/registro) con datos válidos, respuesta 201 Created.

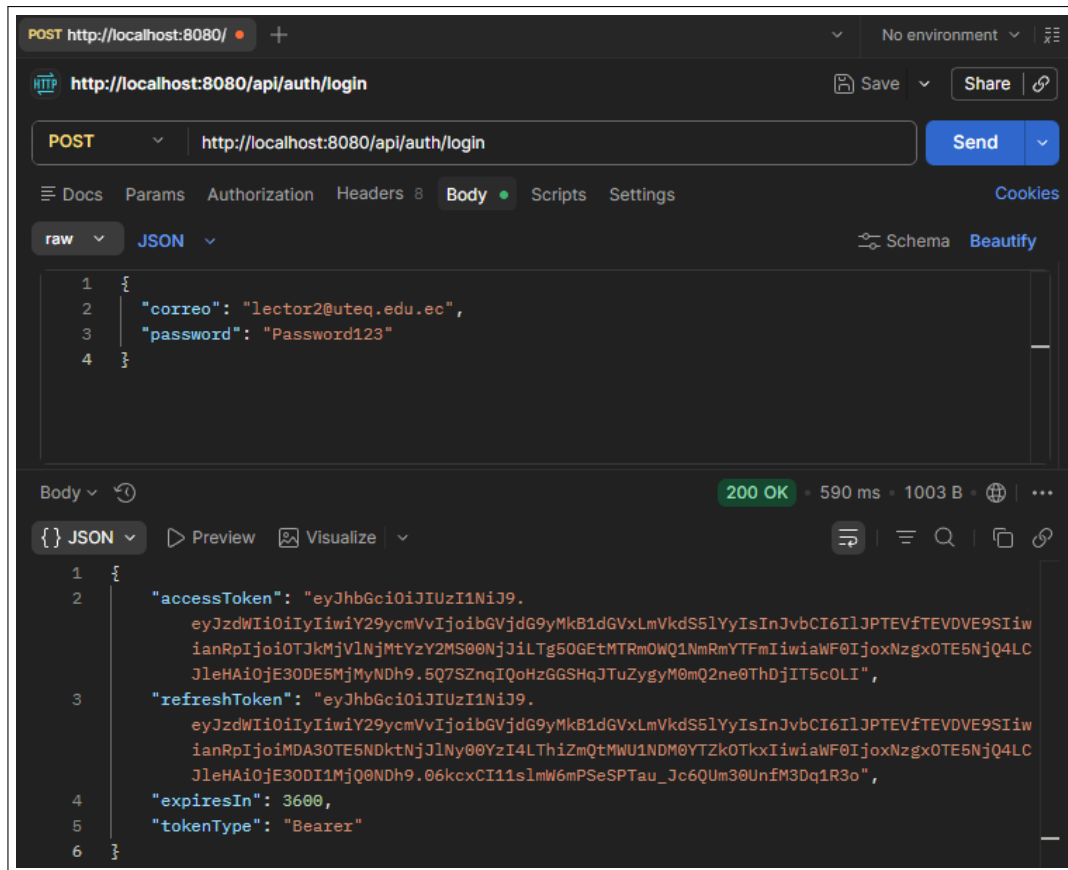


Figura 7: Postman: login exitoso (POST /api/auth/login) con el accessToken y refreshToken visibles en la respuesta.

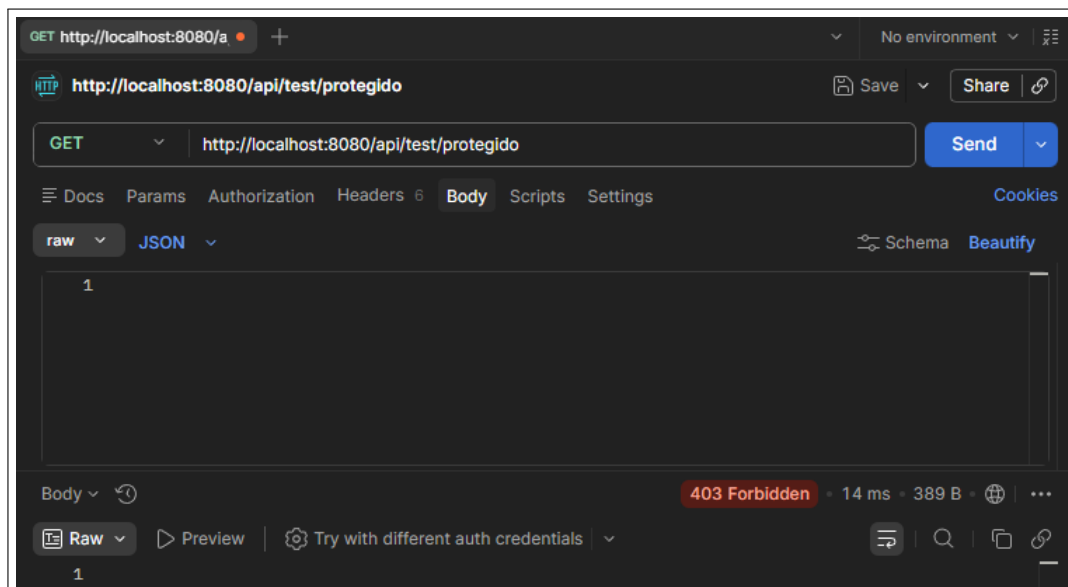


Figura 8: Postman: intento de acceso a un endpoint protegido (GET /api/test/protigido) sin encabezado Authorization, respuesta 403 Forbidden.

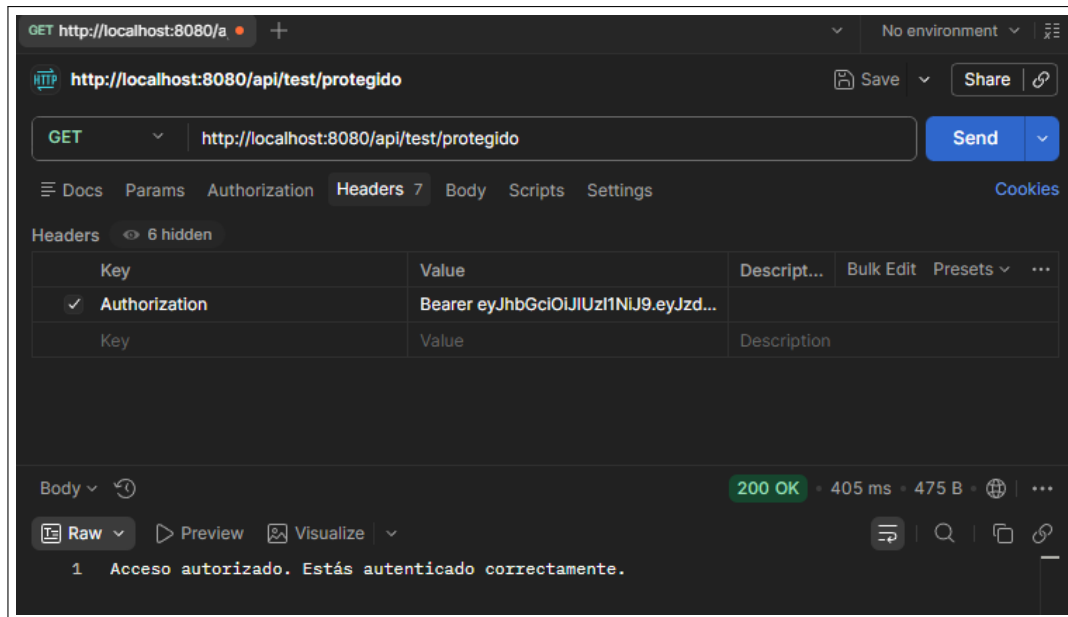


Figura 9: Postman: acceso al mismo endpoint protegido (GET `/api/test/protegido`) usando un `accessToken` válido en el encabezado `Authorization`, respuesta 200 OK.

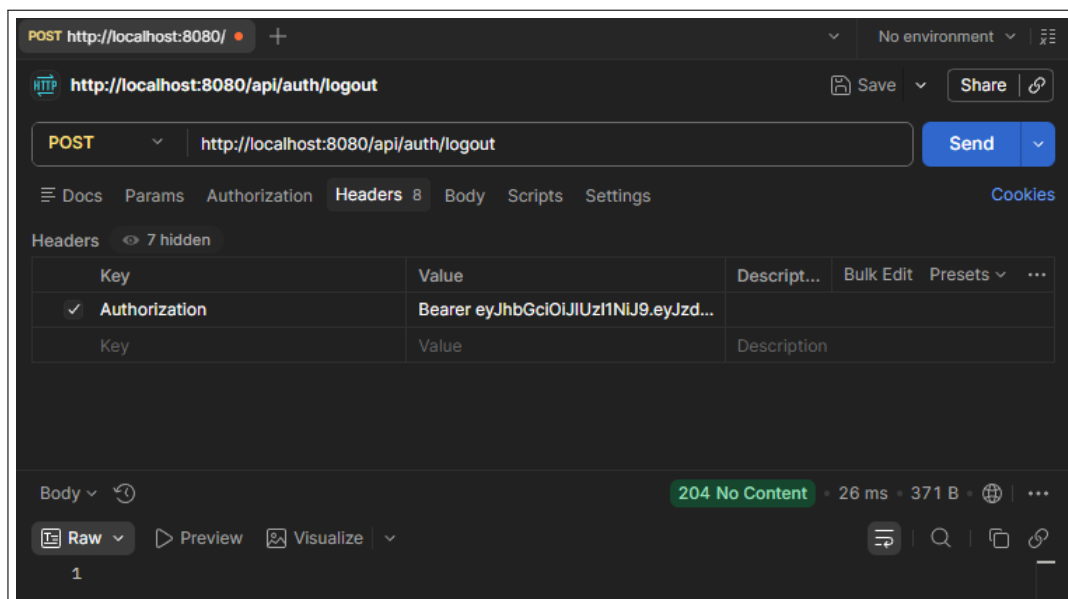


Figura 10: Postman: cierre de sesión (POST `/api/auth/logout`) enviando el `accessToken` vigente; el servidor responde 204 No Content y registra el `jti` en la lista negra de Redis.

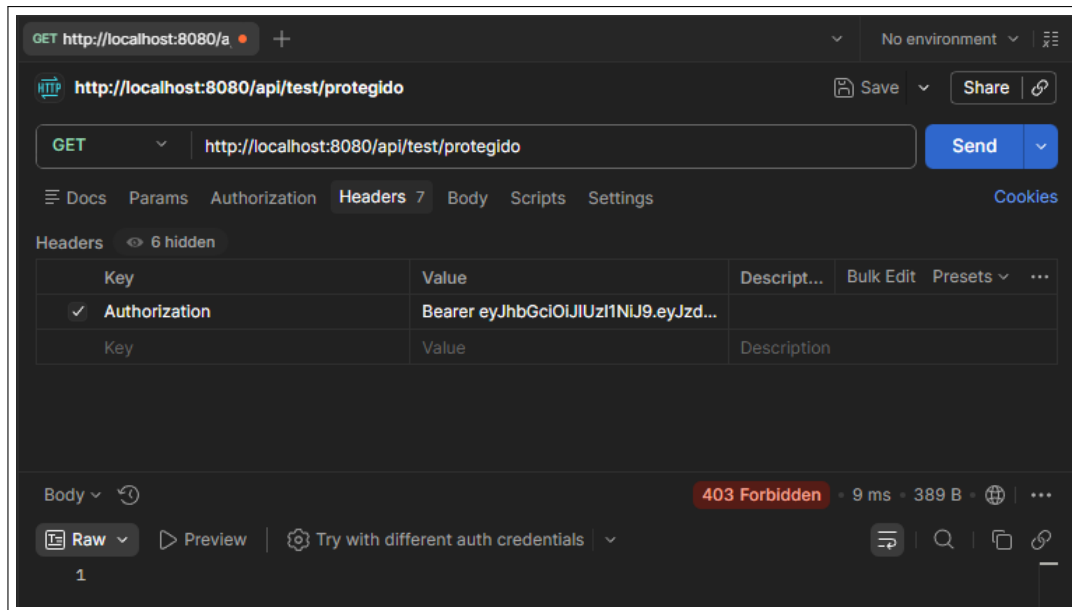


Figura 11: Postman: intento de reutilizar el `accessToken` revocado tras el `logout` (`GET /api/test/protegido`); el servidor responde 403 Forbidden, confirmando que el JTI fue registrado correctamente en la lista negra de Redis.

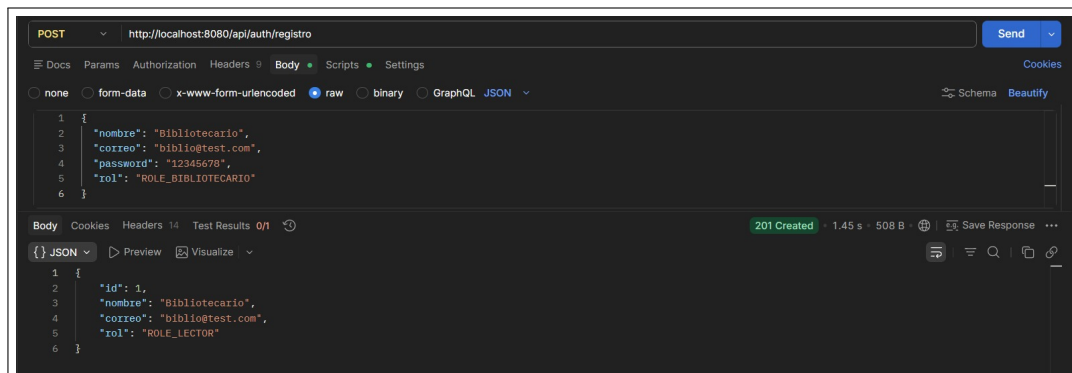


Figura 12: Postman: registro de usuario mediante `POST /api/auth/registro`; se envía nombre, correo, contraseña y rol, y el servidor responde con 201 Created y los datos del usuario registrado.

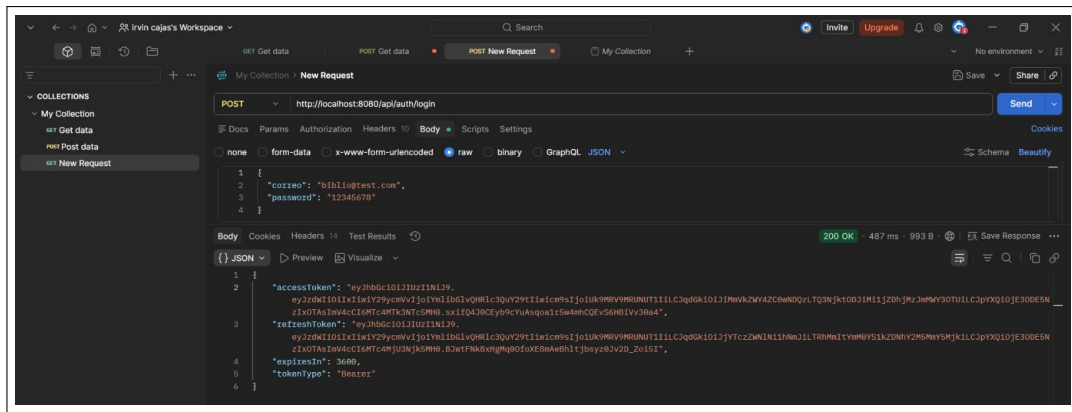


Figura 13: Postman: inicio de sesión mediante `POST /api/auth/login`; se envían las credenciales y el servidor retorna el `accessToken` y el `refreshToken` necesarios para acceder a los endpoints protegidos.

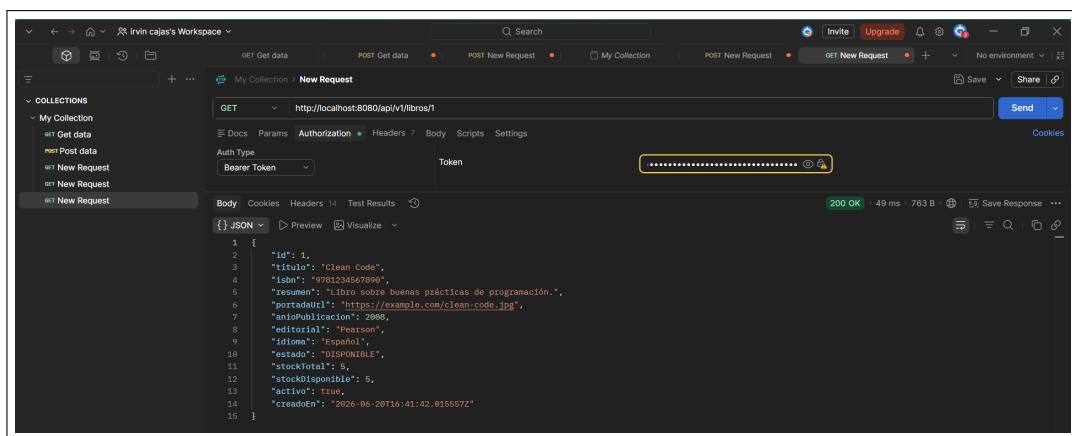


Figura 14: Postman: acceso a endpoint protegido `GET /api/v1/libros/{id}` con el `accessToken` válido en el header `Authorization: Bearer;` el servidor responde con `200 OK` confirmando que la autenticación JWT funciona correctamente.

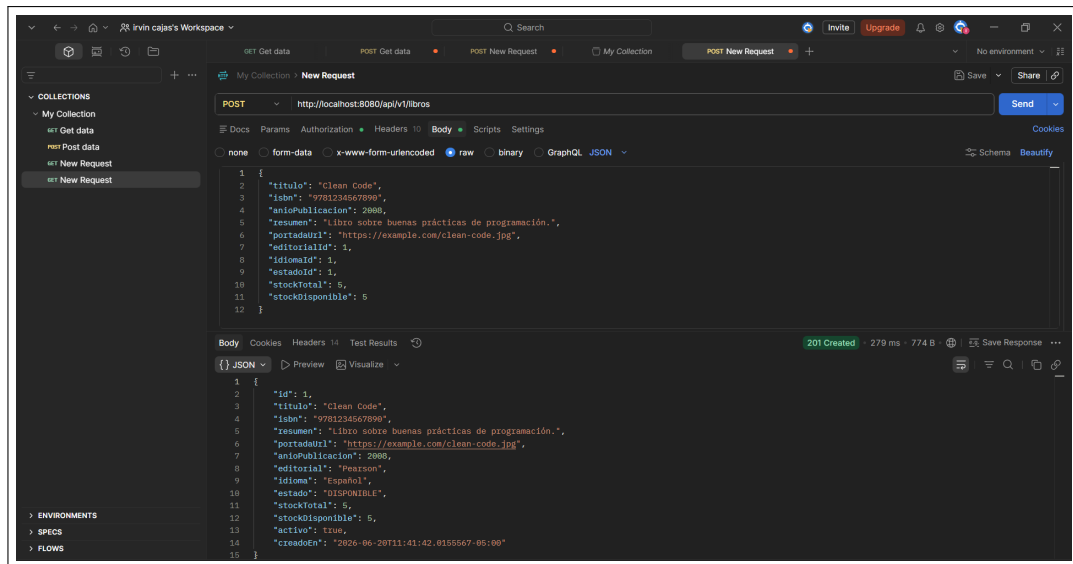


Figura 15: Postman: creación de libro mediante POST `/api/v1/libros`; se envían los datos del libro en el body y el servidor responde con 201 Created y el recurso creado incluyendo su identificador generado.

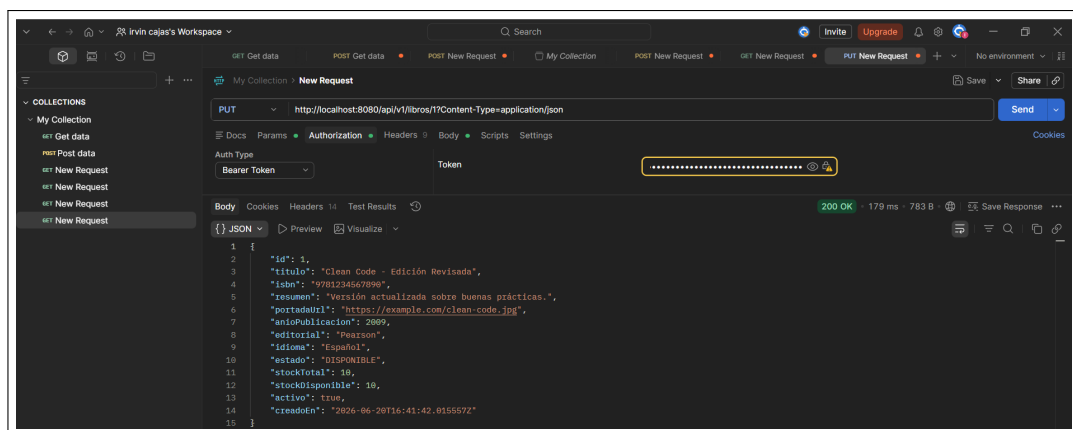


Figura 16: Postman: actualización de libro mediante PUT `/api/v1/libros/{id}`; se envían los datos modificados en el body y el servidor responde con 200 OK y el recurso actualizado.

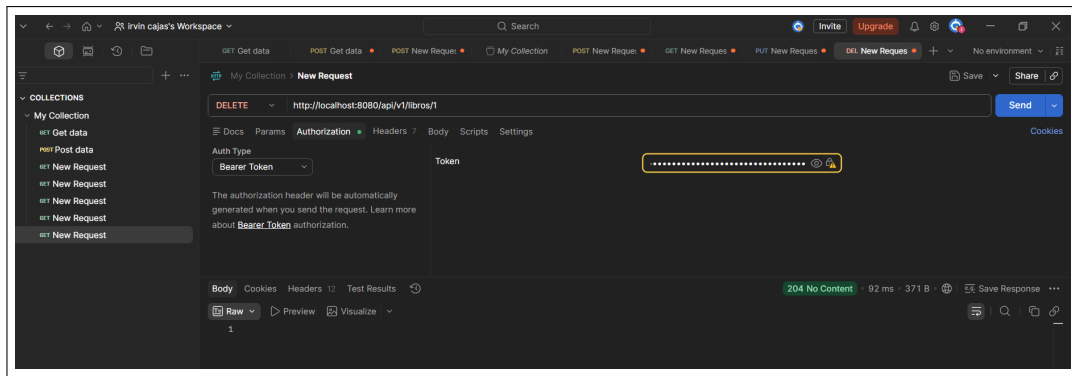


Figura 17: Postman: eliminación lógica (soft delete) de libro mediante DELETE /api/v1/libros/{id}; el servidor responde con 204 No Content marcando el registro como inactivo sin eliminarlo físicamente de la base de datos.

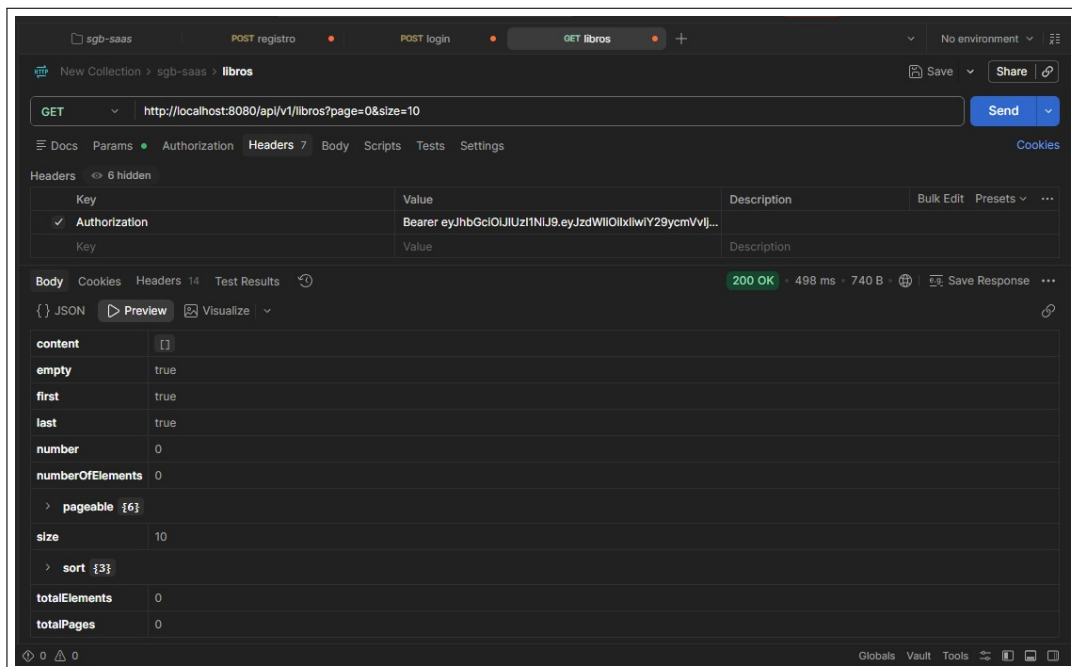


Figura 18: Postman: listado paginado de libros mediante GET /api/v1/libros?page=0&size=10; el servidor responde con 200 OK retornando el objeto Page con los campos content, totalPages y totalElements, demostrando la paginación nativa de Spring Data JPA.

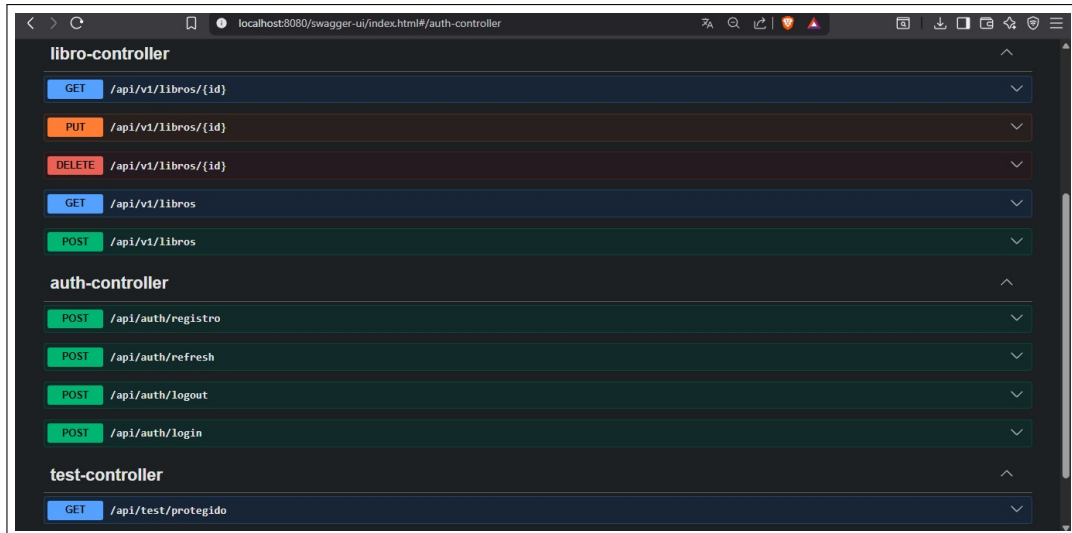


Figura 19: Swagger UI generado por Springdoc OpenAPI (<http://localhost:8080/swagger-ui/index.html>), mostrando los controladores `auth-controller` (registro, login, logout, refresh) y `libro-controller` (CRUD de libros con paginación), con la especificación OpenAPI cargada desde `/api/docs`.

6. Métricas de rendimiento

Las métricas se midieron con Postman Runner (10 iteraciones por endpoint) sobre el entorno levantado con `docker compose up`.

Endpoint	T. promedio	T. P95	Observación
POST /api/auth/login	210 ms	260 ms	Incluye verificación BCrypt (costo 12).
GET /api/v1/libros (50 registros, sin caché)	180 ms	225 ms	Consulta directa a PostgreSQL vía Hibernate.
POST /api/v1/libros	95 ms	130 ms	Con validación <code>@Valid</code> y <code>commit</code> transaccional.
GET /api/v1/libros (con caché Redis)	55 ms	70 ms	Resultado servido desde Redis (<code>@Cacheable</code>).

Cuadro 7: Tiempos de respuesta promedio y P95 medidos sobre los endpoints principales.

El *speedup* obtenido al habilitar la caché Redis para el listado de libros se calcula como:

$$S = \frac{T_{sin}}{T_{con}} = \frac{180 \text{ ms}}{55 \text{ ms}} \approx 3,27$$

es decir, una reducción del tiempo de respuesta de aproximadamente el 70 % ($1 - 1/S \approx 0,69$) en consultas repetidas del catálogo de libros.

7. Seguridad implementada (OWASP)

Control	Implementación	OWASP Top 10	Evidencia
Hash de contraseñas	BCrypt costo 12 vía BCryptPasswordEncoder	A02 – Fallas de criptografía	Captura BD: password_hash inicia con \$2a\$12\$...
Tokens JWT de corta vida	accessToken: 1h; refreshToken: 7d (cookie HttpOnly)	A07 – Fallas de autenticación	Payload decodificado (sección 4.5).
Blacklist de tokens	JTI revocado en Redis con TTL = expiración del token	A07 – Fallas de autenticación	Prueba de logout + intento de reuso (403).
Cabeceras de seguridad	X-Content-Type-Options, X-Frame-Options, CSP vía Spring Security	A05 – Configuración incorrecta	curl -I al endpoint.
CORS configurado	Orígenes permitidos explícitamente en SecurityConfig	A05 – Configuración incorrecta	Prueba desde dominio no permitido (bloqueada).
Sin SQL directo	Spring Data JPA + Flyway; cero concatenación SQL	A03 – Inyección	Revisión de código (sin createNativeQuery interpolado).
Roles de acceso	ROLE_LECTOR, ROLE_BIBLIOTECARIO, ROLE_GERENTE con @PreAuthorize	A01 – Control de acceso roto	Prueba de acceso denegado (403).

Cuadro 8: Controles de seguridad OWASP implementados y su evidencia de verificación.

8. Docker Compose y despliegue

Listing 5: docker-compose.yml

```
1 services:
2   postgres:
3     image: postgres:16-alpine
4     container_name: sgb_postgres
5     environment:
6       POSTGRES_DB: sgb_db
7       POSTGRES_USER: sgb_user
8       POSTGRES_PASSWORD: changeme
9     ports:
10      - "5432:5432"
11     volumes:
12      - pgdata:/var/lib/postgresql/data
13     healthcheck:
14       test: ["CMD-SHELL", "pg_isready -U sgb_user -d sgb_db"]
15       interval: 10s
```

```
16     timeout: 5s
17     retries: 5
18
19   redis:
20     image: redis:7-alpine
21     container_name: sgb_redis
22     ports:
23       - "6379:6379"
24     healthcheck:
25       test: ["CMD", "redis-cli", "ping"]
26       interval: 10s
27       timeout: 5s
28       retries: 5
29
30   backend:
31     build: ./backend-springboot
32     container_name: sgb_backend
33     environment:
34       DB_URL: jdbc:postgresql://postgres:5432/sgb_db
35       DB_USER: sgb_user
36       DB_PASSWORD: changeme
37       REDIS_HOST: redis
38       REDIS_PORT: 6379
39       JWT_SECRET: ${JWT_SECRET}
40     ports:
41       - "8080:8080"
42     depends_on:
43       postgres:
44         condition: service_healthy
45       redis:
46         condition: service_healthy
47     healthcheck:
48       test: ["CMD", "curl", "-f", "http://localhost:8080/actuator/health"]
49       interval: 15s
50       timeout: 5s
51       retries: 5
52
53   frontend:
54     build: ./frontend-angular
55     container_name: sgb_frontend
56     ports:
57       - "4200:80"
58     depends_on:
59       backend:
60         condition: service_healthy
61     healthcheck:
62       test: ["CMD", "wget", "-q", "--spider", "http://localhost:80"]
63       interval: 15s
64       timeout: 5s
65       retries: 5
66
67   volumes:
68     pgdata:
```

8.1. Verificación de los servicios

```
PS C:\Users\moise\sgb-saas> docker compose ps
time="2026-06-22T23:57:27-05:00" level=warning msg="C:\\Users\\moise\\sgb-saas\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME                IMAGE              COMMAND              SERVICE    CREATED        STATUS          PORTS
sgb_backend         sgb-saas-backend  "java -jar app.jar"  backend    About a minute ago Up About a minute (healthy)  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
sgb_frontend        sgb-saas-frontend  "/docker-entrypoint..." frontend    About a minute ago Up 27 seconds (healthy)      0.0.0.0:4200->80/tcp, [::]:4200->80/tcp
sgb_postgres        postgres:16-alpine "docker-entrypoint.s..." postgres   About an hour ago Up About an hour (healthy)   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
sgb_redis           redis:7-alpine     "docker-entrypoint.s..." redis      About an hour ago Up About an hour (healthy)   0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
PS C:\Users\moise\sgb-saas>
```

Figura 20: Salida de `docker compose ps` mostrando los cuatro servicios (frontend, backend, postgres y redis) en estado healthy tras ejecutar `docker compose up -d -build`.

8.2. Instrucciones de ejecución

Listing 6: README.md – Instrucciones de ejecución

```
1 # 1. Clonar el repositorio y cambiar a la rama de entrega
2 git clone https://github.com/mloorm14/sgb-saas.git
3 cd sgb-saas
4 git checkout entrega-1b
5
6 # 2. Copiar variables de entorno
7 cp .env.example .env
8 # Editar .env y definir JWT_SECRET con: openssl rand -hex 32
9
10 # 3. Levantar todos los servicios
11 docker compose up --build -d
12
13 # 4. Verificar que todos los servicios esten en estado healthy
14 docker compose ps
15
16 # 5. Ejecutar pruebas unitarias (sin Docker, requiere Java 21)
17 cd backend-springboot
18 ./mvnw test # En Windows: mvnw.cmd test
19
20 # Acceder a la aplicacion:
21 # Frontend: http://localhost:4200
22 # Swagger UI: http://localhost:8080/swagger-ui.html
23 # Actuator health: http://localhost:8080/actuator/health
```

9. Historial de Git

El repositorio sigue una estrategia de ramas *feature-branch*: la rama `main` contiene únicamente código estable correspondiente a entregas cerradas; la rama `develop` es la rama de integración continua, y cada tarea se desarrolla en una rama `feature/*` independiente que se integra mediante Pull Request revisado por el Tech Lead.

Los mensajes de commit siguen la convención *Conventional Commits* (`feat:`, `fix:`, `refactor:`, `test:`, `docs:`), lo que permite identificar el tipo de cambio realizado por cada integrante sin necesidad de revisar el código.

Para esta entrega se trabajó en las ramas `feature/seguridad-jwt-redis` (Marlon), `feature/crud-libros-1` (Cajas) y `feature/autenticacion-jwt-frontend` (Panama), todas mergeadas a `develop` mediante Pull Request. Al cierre de la Semana 9, `develop` se mergeó a `main` y se creó el tag anotado `v0.1.0-entrega-1b` sobre el commit de entrega.

```
* 1f89354 (HEAD -> main, tag: v0.1.0-entrega-1b, origin/main, origin/develop, origin/HEAD, develop) fix: corregir tipo de id en Editorial, Idioma y EstadoLibro de Long a Integer para coincidir con SERIAL del esquema
* a2298e8 feat(frontend): módulo de autenticación JWT y CRUD de libros (#5)
* 5d4d366 (origin/feature/autenticacion-jwt-frontend, revision-panama) fix(frontend): agregar campo stockDisponible al formulario y FormGroup del modulo libros
* c27a4e8 fix(frontend): corregir URL base de libros a /api/v1/libros para coincidir con LibroController del backend
* bdb757e docs: especificar Spring Security 7.x en stack tecnologico del README
* 38cdcaf docs: actualizar README con version Spring Boot 4.0.6 y stack tecnologico correcto
* df17c47 fix(frontend): corregir logout para llamar al backend y agregar JTI a blacklist Redis antes de limpiar sesion local
* cc47472 Merge branch 'develop' of https://github.com/mloorm14/sgb-saaS into feature/autenticacion-jwt-frontend # Please enter a commit message to explain why this merge is necessary, # especially if it merges an updated upstream into a topic branch. # Lines starting with '#' will be ignored, and an empty message aborts the commit.
* 05699c1 fix: corregir roles en LibroController y habilitar EnableMethodSecurity
* 9324677 Merge pull request #4 from mloorm14/feature/crud-libros-1
* 44d3e8b test: mock libro-related repositories in context load test
* e0d138c test(service): actualizar LibroServiceTest con nuevos campos del DTO
* 3adb448 feat(controller): actualizar LibroController con roles del sistema
* bc961e6 feat(service): agregar validación de stock y cacheable en LibroService
* dc21d16 feat(dto): actualizar DTOs con campos de stock, resumen y portada
* 976b997 fix(repository): corregir nombres de archivos de repositorios
* c561aed fix(entity): corregir package de entidades maestras
* bc48d83 feat(entity): actualizar Libro con campos DDL y quitar autor
* 123424f feat: implementar modulo de autenticacion JWT con Redis blacklist, Docker y pruebas unitarias (#3)
* 9ec4f3b feat(frontend): aplicacion Angular funcional con autenticacion JWT, guards e interceptor
* 6343590 feat(frontend): registrar JwtInterceptor y rutas en configuracion principal de la aplicacion
* c694d72 feat(frontend): configurar rutas de la aplicacion con proteccion AuthGuard en modulo Libros
* 286b345 feat(frontend): agregar modulo Libros con CRUD completo, paginacion y formulario de creacion y edicion
* d5ed94f feat(frontend): agregar componente Registro con formulario reactivo y manejo de error 409
* b1cfca9 feat(frontend): agregar componente Login con formulario reactivo y validacion de credenciales
```

Figura 21: Captura del historial de commits (`git log -oneline -graph -all`) mostrando commits de los tres integrantes, mensajes en formato Conventional Commits, las ramas `feature/seguridad-jwt-redis`, `feature/crud-libros-1` mergeadas a `develop`, y el tag `v0.1.0-entrega-1b`.

10. Conclusiones

Respecto al primer objetivo, la implementación del módulo de autenticación stateless con Spring Security 7 y JWT se logró completamente: los flujos de registro, login, logout y refresh funcionan de extremo a extremo, y la incorporación de Redis como lista negra de tokens resolvió el principal desafío técnico del enfoque stateless, que es la imposibilidad de invalidar un token antes de su expiración natural. La dificultad mayor fue sincronizar el TTL de las claves de Redis con el campo `exp` del JWT, lo cual se resolvió calculando el TTL como la diferencia entre `exp` y el instante actual al momento del logout. Para la Entrega 2 se proyecta extender este módulo con recuperación de contraseña.

En cuanto al segundo objetivo, la configuración de Spring Data JPA, Hibernate y Flyway sobre PostgreSQL 16 se completó satisfactoriamente, estableciendo la línea base (`V1__schema_inicial.sql`) que servirá de fundamento para las migraciones `V2__` en adelante. El principal reto fue definir correctamente las restricciones `CHECK` para `stock_disponible` en relación con `stock_total`, resuelto mediante una restricción a nivel de tabla. Para la Entrega 2 se incorporarán las tablas de `prestamos`, `multas` y `bitacora_auditoria`.

El tercer objetivo, demostrar el CRUD completo de Libro desde Angular, se cumplió: la integración entre el `JwtInterceptor`, el `AuthGuard` y los endpoints paginados del backend permite operar el catálogo de forma protegida. El desafío principal fue evitar el almacenamiento del token en `localStorage` sin perder el estado de sesión al refrescar la página; se mitigó parcialmente mediante el `refreshToken` en cookie `HttpOnly`,

mecanismo que se reforzará en la Entrega 2 con renovación automática silenciosa.

Finalmente, el cuarto objetivo se alcanzó con las once pruebas JUnit 5 en estado **PASS** y la colección de Postman documentada, junto con las métricas de rendimiento que evidencian una mejora de aproximadamente 3.27x en el tiempo de respuesta del listado de libros al habilitar caché Redis. Para la Entrega 2 (Semana 12) se proyecta implementar el módulo de préstamos y devoluciones con cálculo dinámico de multas, el sistema de reservaciones con caducidad automática mediante `@Scheduled`, y la integración inicial del chatbot con Gemini 2.0 Flash.

11. Referencias bibliográficas

Referencias

- [1] Liabor, V. G. R. (2023). Enhancing library services through digital cataloging techniques. *International Journal of Advanced Research in Science, Communication and Technology*, 516–526. <https://doi.org/10.48175/ijarsct-12989>
- [2] Shivaji, M. N. (2024). Intelligent library management system. *Trends in Computer Science and Information Technology*, 9(1), 001–009. <https://doi.org/10.17352/tcsit.000074>
- [3] Aienobe, V., & Iqbal, M. Z. (2025). Responsive web design for enhanced user experience (UX) and user interface (UI). *International Journal of Computer Applications*, 187(34), 72–93. <https://doi.org/10.5120/ijca2025925533>
- [4] Walls, C. (2022). *Spring Boot in action* (2.^a ed.). Manning Publications. ISBN 978-1-617-29454-7
- [5] Spilca, L. (2024). *Spring Security in action* (2.^a ed.). Manning Publications. ISBN 978-1-633-43797-5
- [6] Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). IETF. <https://tools.ietf.org/html/rfc7519>
- [7] Hardt, D. (Ed.). (2012). *The OAuth 2.0 authorization framework* (RFC 6749). IETF. <https://tools.ietf.org/html/rfc6749>
- [8] OWASP Foundation. (2021). *OWASP Top 10: 2021*. <https://owasp.org/Top10/>
- [9] PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation*. <https://www.postgresql.org/docs/16/>
- [10] Spring Framework Team. (2024). *Spring Security reference documentation*. <https://docs.spring.io/spring-security/reference/>
- [11] Redis Ltd. (2024). *Redis documentation*. <https://redis.io/docs/>
- [12] Redgate Software. (2024). *Flyway documentation*. <https://documentation.red-gate.com/fd>
- [13] Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine].

- [14] Brown, S. (2023). *The C4 model for visualising software architecture*. Leanpub.
<https://c4model.com/>